21 不相交集合

Part V Advanced Data Structures

本章定位

不相交集合把只合不分的partition轉成代表查詢；本章以同一組合併對照weighted list與rank forest，再由完整level/iter位能計費推導反Ackermann總界，區分單次成本、序列攤銷與實作前提。

前置知識

Representative、森林、rank 與路徑壓縮。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 比較 linked-list 與 forest 表示。
- 用 union by rank 與 path compression。
- 理解 inverse Ackermann 攤銷界。
- 21.1 Disjoint-set operations
- 21.2 Linked-list representation of disjoint sets
- 21.3 Disjoint-set forests
- 21.4 Analysis of union by rank with path compression

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21.1 Disjoint-set operations

維護的是 partition，代表元素只是 handle

Universe中的每個已建立元素恰屬一個非空集合，集合兩兩不交。MAKE-SET(x)要求x尚未建立，產生{x}；FIND-SET(x)要求x存在，回傳其集合的一個代表；UNION(x,y)將兩個集合合成一個。代表不自動等於最小鍵，內部parent亦不是原圖的邊。

本章令n為MAKE-SET次數，m為包含MAKE-SET、FIND、UNION的全部API呼叫數，m≥n；為固定分析參數，先完成n個單點再操作。成功合併最多n−1次，但重複UNION API可任意多。實作先取兩代表，若相同直接返回，不得把同一根重新LINK或增加rank。教材採此可重複呼叫的契約，LINK則只接受**不同的兩個根**。

FIND可能壓縮parent但不改partition；UNION只合不分。刪除元素、撤銷合併、動態刪圖邊或輸出全部成員，均不包含於三操作的既有保證，須另設結構。不同集合只要求代表不同，同一集合的所有FIND必回同一代表。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連通分量：為何合併後既不漏也不誤合

對無向圖，先MAKE-SET每個頂點；每來一條(u,v)就UNION(u,v)。不變量：處理的邊所形成子圖，其連通分量恰等於DSU partition。初始只有單點；新邊兩端同分量時兩者都不變，不同分量時恰用該邊接成一個分量，所以歸納成立。結束後FIND(u)=FIND(v)⇔存在原圖無向u-v路。

六點a..f依序加入ab、cd、bc、ab、ef、de：分量依序{ab},{c},{d},{e},{f}；{ab},{cd},{e},{f}；{abcd},{e},{f}；重複ab不變；{abcd},{ef}；{abcdef}。成功合併5次，API UNION有6次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連通分量：為何合併後既不漏也不誤合 （續）

邊串流：partition 的每次改變

每列為處理新邊後的partition；第4列重複邊不改集合數。

			集合數
ab	⇒	{a,b} {c} {d} {e} {f}	5
cd	⇒	{a,b} {c,d} {e} {f}	4
bc	⇒	{a,b,c,d} {e} {f}	3
ab	⇒	{a,b,c,d} {e} {f}	3
ef	⇒	{a,b,c,d} {e,f}	2
de	⇒	{a,b,c,d,e,f}	1

成功5次合併：6→5→4→3→3→2→1；全部6次 UNION 仍各需做代表檢查。

靜態圖DFS/BFS已可Θ(n+|E|)找分量；DSU的用途還包括邊逐一加入時即時查連通，或Kruskal辨別新邊是否成cycle。有向圖若忽略方向合併只得到弱連通，不是SCC。

21.2 Linked-list representation of disjoint sets

Set record 與每個成員的反向指標

每集合record存head、tail、size；每成員存next及set指標。代表是record.head的成員。空collection可O(1)建立；MAKE-SET產生單一節點，next=NIL、head=tail=x、size=1、x.set=新record。FIND只回x.set.head，O(1)。

UNION取得S=x.set、T=y.set；若相同返回head。若S.size<T.size交換S,T，同大保留第一個S。將S.tail.next=T.head、S.tail=T.tail、S.size+=T.size，再逐一走T原串列，把每個成員的set改成S，最後廢棄舊record T。串列next順序未在走訪中改變，因此不會遺失後繼；不能只接tail卻不更新set。

一次UNION成本Θ(1+min(|S|,|T|))，不是O(1)。例如長串列a→b→c、短串列d→e，合併後a→b→c→d→e，僅d,e的set改指長record；tail由c改e、size由3改5，d/e的FIND才會回a。

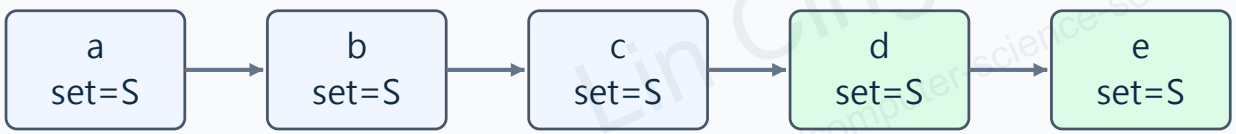
21.2 Linked-list representation of disjoint sets （續）

Weighted list：接 next 之外還要改 set

合併前 S: head=a, tail=c, size=3；T: head=d, tail=e, size=2。



短串列接到長尾；只改 d,e 的 set → S，並令 c.next=d。



合併後 S: head=a, tail=e, size=5；綠格 = 本次改set的成員；箭頭 = next。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不加權的平方反例與加權的 doubling 證明

若總把第二個集合接到第一個尾端，先n個單點，再做UNION(x_2,x_1)、UNION(x_3,x_2)、…，每次新的單點吸收之前整個長串列。改寫成員數依序1,2,…,n−1，合計n(n−1)/2；m=2n−1卻需Θ(n^2)，所以「有tail可O(1)串接」不等於整個UNION O(1)。

Weighted union每次只移短集合。固定成員x每次set改寫，所在集合大小至少倍增；改k次後至少2^k個成員，最多n，故k≤floor($\log_2 n$)。加總n個成員最多n floor($\log_2 n$)次改寫；每次API常數開銷另計m，總O(m+n log n)，小n用log(n+1)。單次兩個各n/2集合合併仍Θ(n)，不能把此總界解讀成每次最壞O(log n)。

上界亦有對應成長序列：$n=2^k$時，每輪成對合併同大小集合，k輪每輪恰改寫n/2個set，共(n/2)$\log_2 n$。MAKE/FIND的O(1)與整段總成本可以同時成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八元素的每次改寫帳

同大小保留第一集合。下列次序亦供森林版本使用；字母只是handle，不作大小比較。

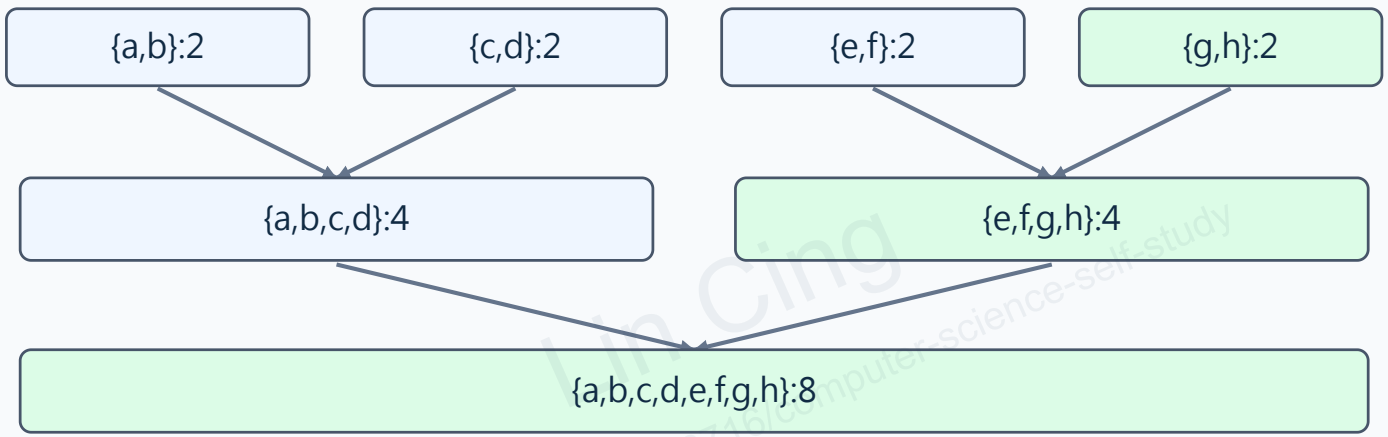
UNION	原大小	改set的成員	新代表／大小	累計改寫
a,b	1,1	b	a／2	1
c,d	1,1	d	c／2	2
e,f	1,1	f	e／2	3
g,h	1,1	h	g／2	4
a,c	2,2	c,d	a／4	6
e,g	2,2	g,h	e／4	8
a,e	4,4	e,f,g,h	a／8	12

h的set依h→g→e→a，改寫時新集合大小2→4→8，恰三次；a完全不必改。所有成員的次數加總12，並非每個成員都三次。若總以單點接到大集合的加權版本，前述平方反例會變成只有n−1次成員改寫。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

成對同大合併：三輪的逐人改寫帳

每輪改寫4個set；綠框是 h 所在集合，大小2→4→8。



三輪總改寫 4+4+4=12；h改3次，a改0次；集合不分裂。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Root 自指與 LINK 的前提

每元素存parent、rank，根parent指向自己。MAKE-SET令parent[x]=x、rank[x]=0。UNION不是把任意x直接接到y，而是先找兩根；否則可能切斷原集合或製造cycle。

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    若 r==s: 回傳 r
    回傳 LINK(r,s)
LINK(r,s):                      // r,s 是不同的根
    若 rank[r]<rank[s]: 交換 r,s
    parent[s]=r
    若 rank[r]==rank[s]: rank[r]+=1
    回傳 r
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

接根後整個舊集合仍沿parent到新根；只有一個root失去自指，沒有反向可回的路，所以仍為森林。不同rank不增加新根rank；相同rank只使保留下來的新根加一。這是union by rank，與linked list按size比較是兩種不同表示法，不能把rank當精確成員數。

兩趟 FIND：覆寫前保存舊父

```
FIND(x):
    r=x
    當 parent[r]!=r: r=parent[r]
    當 parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    回傳 r
```

第一趟找根而不改任何指標；第二趟走原路，把沿途parent全部改r。若先覆寫parent[x]再用它繼續走，就會直接跳r而漏壓縮中間節點。遞迴寫法parent[x]=FIND(parent[x])等價於上行找根、返回時改連結；遞迴stack最多路徑高度，兩趟迭代輔助O(1)。

壓縮只把同樹節點改接其祖先根，根與partition都不變；不修改rank。一次走s個節點成本Θ(s)。森林本身O(n)空間，若還要列全部成員，可另維護成員串列；單靠parent不會自動提供O(集合大小)列舉。

同一八元素的 rank 與壓縮前後

沿前述七次UNION，同rank保留第一根，根rank依a1,c1,e1,g1,a2,e2,a3。最終parent為b→a、c→a、d→c、e→a、f→e、g→e、h→g；a自指。

點	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
FIND(h)前parent	a	a	a	c	a	e	e	g
FIND(h)後parent	a	a	a	c	a	e	a	a

第一趟h→g→e→a三跳，第二趟h.parent=a、g.parent=a、e.parent=a（最後一個原本已如此）。d→c與f→e不在此find path，不改；rank全部保持。再FIND(h)只一跳，但FIND(d)仍兩跳，不能把一次find說成全樹扁平化。

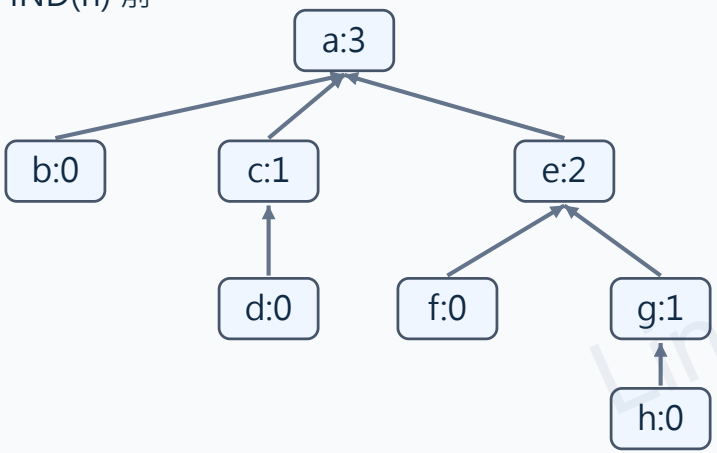
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一八元素的 rank 與壓縮前後 （續）

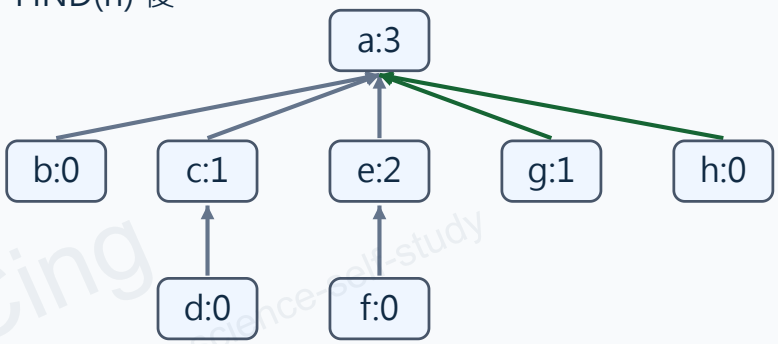
同一八點樹：壓縮前後的完整 parent

節點 = 鍵:rank；箭頭 = parent（根自指省略）；綠線 = 本次改寫。

FIND(h) 前



FIND(h) 後



h→g→e→a；只改 h,g 的實際連線，e 本就指 a；d,f 不在路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank 的高度上界與歷史成員下界

對操作數歸納，root的目前高度≤其rank：單點為0；不同rank時小樹高度+1≤大rank，相同rank時最多原rank+1；壓縮只會縮短路徑，不增加高度。根rank=r的集合至少2^r個成員：只在兩個rank r−1根合併時升級，兩集合各至少2^{r-1}；其他合併只增加成員。非根的rank固定，沿用它最後為根時所在集合的歷史下界，因此任一rank≤floor($\log_2 n$)。

Rank不是目前高度的下界。例中a.rank=3，但壓縮全部元素後根只有高度1；rank=2的根亦能在四點星狀結構中高度1。它也不是目前subtree大小，因壓縮會移走後代，卻不減rank。

不用壓縮、只by-rank，單次FIND O(log n)，全部m次O(m log n)；成對合併出高度$\log_2 n$，再對深葉重複FIND便達對應量級。沒有by-rank時可先LINK成線性長鏈，使第一次FIND Θ(n)；不能把這種反例套到同時使用兩heuristics的定理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ackermann 的迭代是函數合成

固定整段序列最終的元素數 n，令 $\alpha = \alpha(n)$。分析時不隨目前已建立的元素數重算 α。對 $j \geq 1$，定義

$$f^{(0)}(j) = j, \qquad f^{(i+1)}(j) = f(f^{(i)}(j)), \qquad A_0(j) = j + 1, \quad A_k(j) = A_{k-1}^{(j+1)}(j)\ (k \geq 1).$$

上標表示**反覆把輸出當成下一次輸入**。由 $A_0^{(i)}(j) = j + i$ 得 $A_1(j) = 2j + 1$；對 i 歸納可得 $A_1^{(i)}(j) = 2^i(j+1) - 1$，所以

$$A_2(j) = 2^{j+1}(j+1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$$

各 A_k 對輸入嚴格遞增，且 $A_k(j) \geq j + 1$：由 A_0 開始歸納，較大的輸入會經過更多次、且每次都增加的迭代。又 $j + 1 \geq 2$，故 $A_k(j) > A_{k-1}(j)$。後面的分層界會用到這兩種單調性。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

本章採用的 inverse-Ackermann 定義

採 Cormen 第三版 §21.4 的單參數版本：

$$\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}.$$

k	0	1	2	3	4
$A_k(1)$	2	3	7	2047	大於 10^{80}

所以 $\alpha(2) = 0$、$\alpha(8) = 3$、$\alpha(2048) = 4$。不同教材可能採雙參數定義；不能直接混用其數值。

完整總界是 $O(m(1 + \alpha(n)))$，其中 m 次外部操作包含 n 次 MAKE-SET。只有 $n \geq 3$ 時才能簡寫為 $O(m\alpha(n))$；$n \leq 2$ 仍有常數操作成本。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

哪些量會隨操作改變？

對操作數歸納可證：根以 parent 指向自己；非根的 rank 嚴格小於父 rank。相同 rank 的兩根合併時，保留根的 rank 加一；不同 rank 合併時，把小 rank 根接到大 rank 根。壓縮只改 parent，不改 rank。

節點一旦成為非根，其 rank 就固定。 後續它所指父節點的 rank 可能增加，或因壓縮改指更高的祖先；兩者都不會降低父 rank。這才是 level 不下降的依據。

rank 為 r 的根至少包含 2^r 個元素。非根沿用失去根身分時的歷史下界，故任一節點的 rank 至多 $\lfloor \log_2 n \rfloor < n$。這不是壓縮後的目前子樹大小。

接下來只對 rank 正、且不是根的節點定義 level 與 iter；rank 0 和根另外計費。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

level 決定層級，iter 記錄層內進度

令 $r = \operatorname{rank}(x) \geq 1$、$p = \operatorname{rank}(\operatorname{parent}(x))$，定義

$$\ell(x) = \max\{k \geq 0 : p \geq A_k(r)\}, \qquad i(x) = \max\{j \geq 1 : p \geq A^{(j)}_{\ell(x)}(r)\}.$$

正文以 $\ell = \text{level}$、$i = \text{iter}$ 縮短公式。先找父 rank 能跨過的最高層，再算該層可迭代幾次。

因 $p \geq r + 1 = A_0(r)$，有 $\ell \geq 0$。又 $A_\alpha(r) \geq A_\alpha(1) \geq n > p$，故 $\ell < \alpha$。定義給 $i \geq 1$；若 $i \geq r + 1$，就會有

$$p \geq A^{(r+1)}_\ell(r) = A_{\ell+1}(r),$$

與 ℓ 的最大性矛盾。因此 $0 \leq \ell < \alpha$、$1 \leq i \leq r$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用非負位能記錄尚可支付的工作

逐節點定義位能，再加總成整座森林的位能：

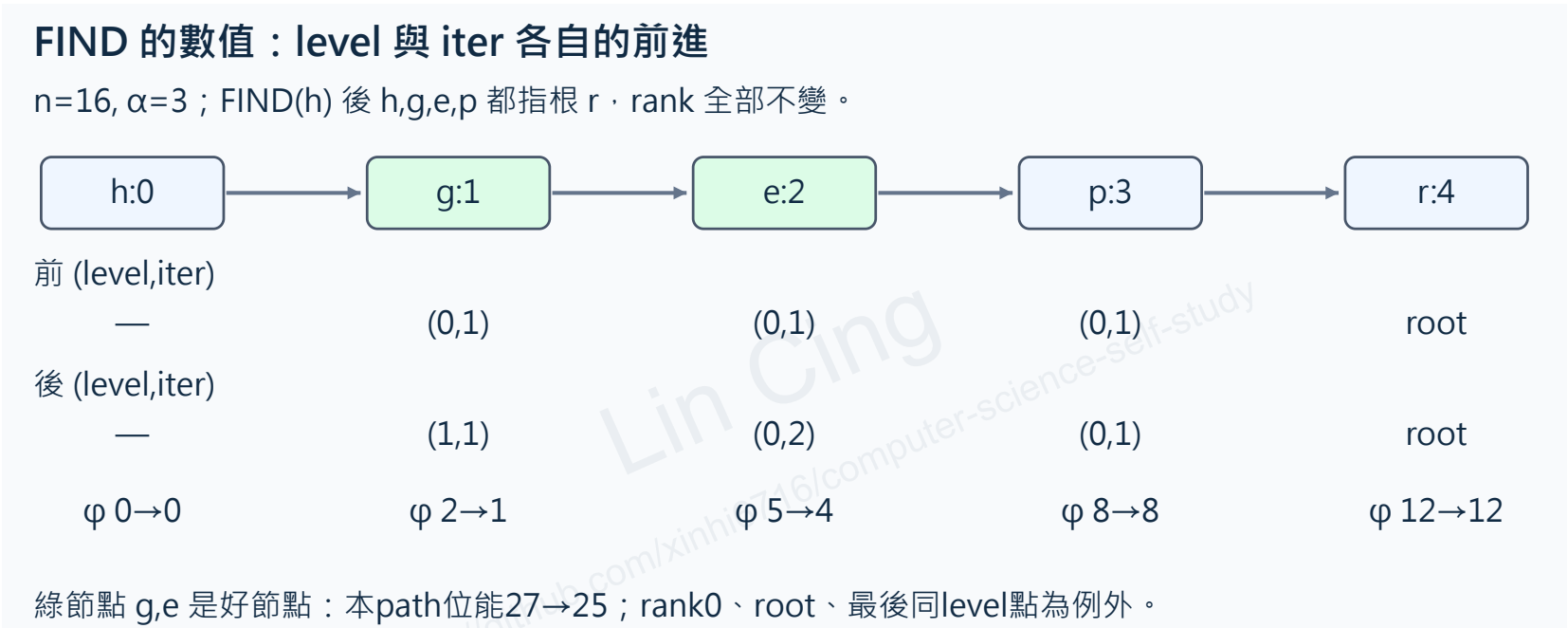
$$\phi(x) = \begin{cases} \alpha r, & x \text{ 是根或 } r = 0, \\ (\alpha - \ell(x))r - i(x), & x \text{ 非根且 } r \geq 1, \end{cases} \qquad \Phi = \sum_{x} \phi(x).$$

非根的下界為 $(\alpha - (\alpha - 1))r - r = 0$，上界為 $\alpha r - 1$。根的位能是 αr，rank 0 的位能為 0；所以 Φ 永遠非負。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

五點路徑的位能帳

取 $n = 16$、$\alpha = 3$，壓縮 rank 依序為 0、1、2、3、4 的路徑。圖中的根保持自指。



兩個好節點各釋放 1：g 靠 level 上升，e 靠同層 iter 上升。其餘三點位能不變，所以整條路徑位能由 27 降至 25。

level 上升時，iter 為何可以下降？

固定非根的 rank r。同一層內，父 rank 增加只會讓 iter 不變或增加；每多一次迭代，ϕ 就減一。

若層級由 k 升到 $k' > k$，iter 可能由 i 重設為較小的 i'。然而

$$\Delta\phi = -(k' - k)r + (i - i') \leq -r + (r - 1) = -1.$$

不等式用了 $k' - k \geq 1$、$i \leq r$、$i' \geq 1$。**跨層的位能下降，足以支付 iter 重設。** 因此需要單調的是「先比較 level，再比較 iter」的字典序（lexicographic order），不是兩個座標各自遞增。

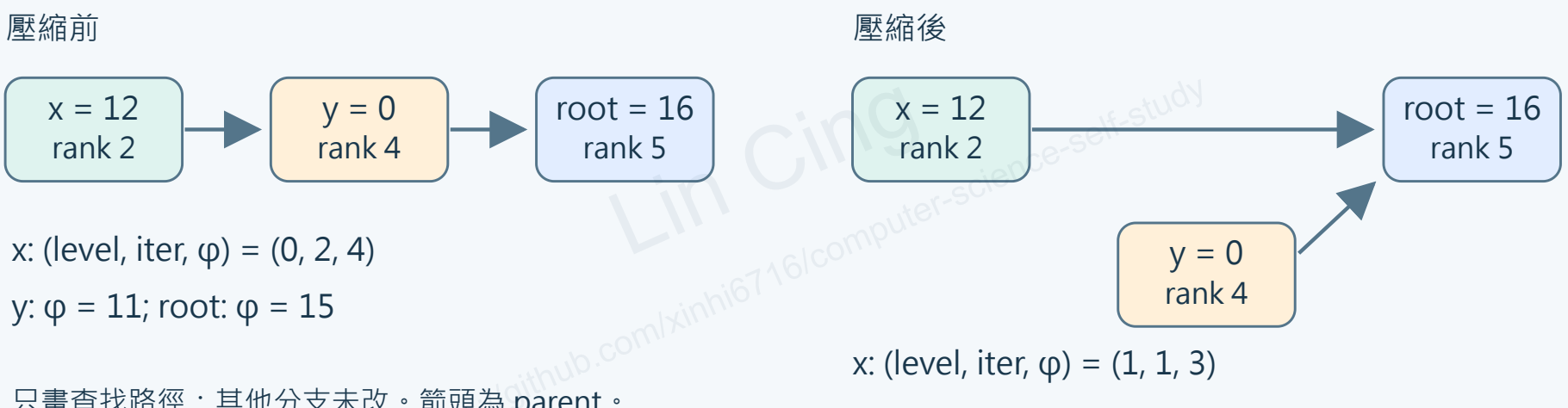
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個 iter 確實下降的合法 FIND

取 $n = 32$、$\alpha = 3$。查找路徑是節點 $12 \to 0 \to 16$，rank 依序為 $2, 4, 5$。圖只保留這次路徑，其他分支不變。

level 上升，iter 仍可由 2 降為 1

n=32, α=3; FIND(12)



Lin Cing https://github.com/xinhi6716/computer-science-self-study

跨層數例的逐項計算

對 $x = 12$，固定 rank 為 2；壓縮只把父 rank 由 4 換成 5。

量	壓縮前	壓縮後
level	$A_0(2) = 3 \le 4 < A_1(2) = 5$，故為 0	$A_1(2) = 5 \le 5 < A_2(2) = 23$，故為 1
iter	$A_0^{(2)}(2) = 4 \le 4 < A_0^{(3)}(2) = 5$，故為 2	$A_1(2) = 5 \le 5 < A_1^{(2)}(2) = 11$，故為 1
$\phi(x)$	$(3-0)2 - 2 = 4$	$(3-1)2 - 1 = 3$

節點 0 的位能保持 11，根的位能保持 15；其餘分支都不變。因此全森林 $\Delta\Phi = (3 + 11 + 15) - (4 + 11 + 15) = -1$。這裡只列路徑的加總；並未假設其他 29 個節點的位能為零。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

如何真的建立這條路徑？

下列只用 MAKE-SET、UNION、FIND；同 rank 保留第一個根。先建立鍵 0 到 31 的單點。

操作	可核對的結果
對 0..15 分輪合併相鄰、同大小區塊	根 0 的 rank 為 4；節點 12 的 rank 為 2
FIND(12)	把 12 的 parent 改為 0；不改 rank
同法合併 16..31	根 16 的 rank 為 4
UNION(16,0)	根 16 升為 rank 5；0 的 parent 為 16
再次 FIND(12)	沿 $12 \to 0 \to 16$；12 的 parent 改為 16

例如前半部的輪次是先合併 (0,1)、(2,3)、…、(14,15)，再合併 (0,2)、(4,6)、…，接著 (0,4)、(8,12)，最後 (0,8)。這說明 rank 數值來自合法歷史，沒有任意指定節點的 rank。

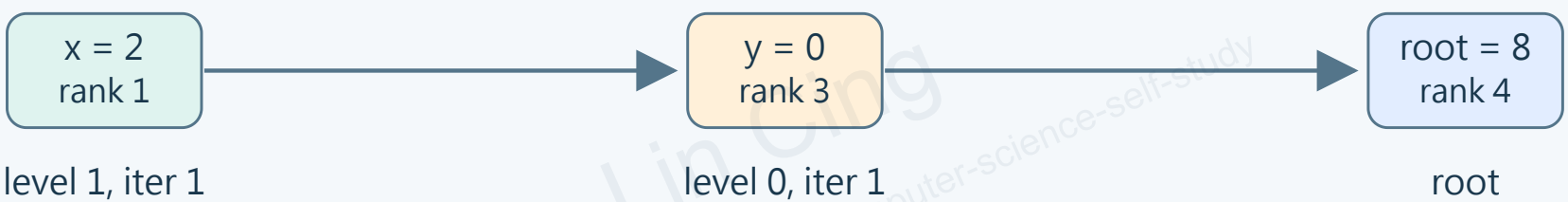
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

沿路的 level 不必遞增

取 $n = 16$。先把 0..7 合成根 0、rank 3 的集合並執行 FIND(2)；再把 8..15 合成根 8、rank 3 的集合，執行 UNION(8,0)。得到以下合法路徑。

rank 遞增，不代表 level 也遞增

n=16, α=3; FIND(2)



$A_1(1)=3 \le \text{rank}(y) < A_2(1)=7$; $A_0(3)=4 \le \text{rank}(\text{root}) < A_1(3)=7$

沿路 rank: 1 < 3 < 4；level: 1 > 0。

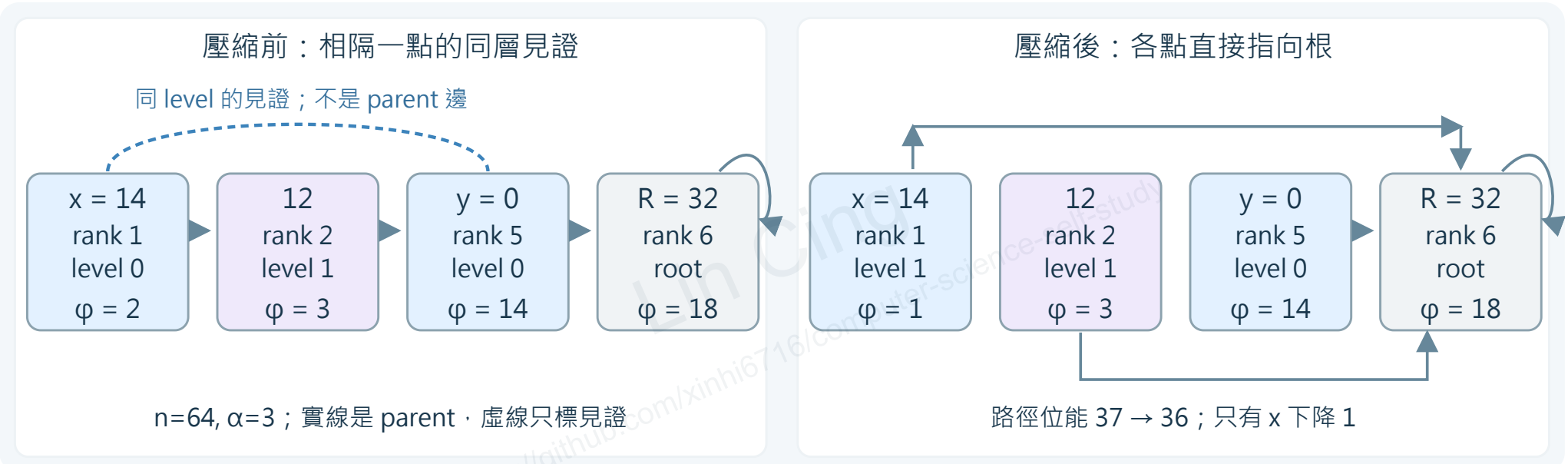
同 level 的最後一點才是例外；不用假設 level 沿路排序。

不同節點的 rank 也變了；不能把沿路比較，混同「同一節點隨時間」的單調性。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

好節點的同層見證可以隔著其他 level

在壓縮前的路徑上，rank 大於 0 的非根節點 x 的後方若有另一個非根 y，且 $\ell(x) = \ell(y)$，就稱 x 為好節點。**找同層的後繼，不是只比較下一個節點。**



路徑 $14 \to 12 \to 0 \to 32$ 的 rank 為 1、2、5、6，level 為 0、1、0（根不定義 level）。14 的見證是 0；中間的 12 不必同層。兩個同層點不必在原樹中相鄰，壓縮後仍能替 14 帶來更高的父 rank。

先用合法合併建立這條路徑

取 64 個單點，同 rank 保留 UNION 的第一個根。「分輪建立區間」表示先合併相鄰單點，再合併相鄰同大小區塊。

操作	能從操作歷史核對的事實
分輪建立 0..31	根 0 的 rank=5；節點 12 的 rank=2，14 的 rank=1
FIND(12)	原路徑 $12 \to 8 \to 0$；只把 12 改接 0，14 仍接 12
分輪建立 32..63	根 32 的 rank=5
UNION(32,0)	兩根同 rank，32 升到 6；0 改接 32
尚未 FIND(14)	路徑為 $14 \to 12 \to 0 \to 32$，rank 為 1、2、5、6

FIND(12) 沿 $12 \to 8 \to 0$ 往上找到根，再把這條路上的父指標直接接到根。它不走進子樹，所以 14 仍接 12。之後的合併才把根 0 接到根 32；這些 rank 都來自合法操作，根 32 保持自指。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

把每個節點的 level、iter、位能算出來

固定 $n = 64$、$\alpha = 3$。只對rank 大於 0 的非根節點計算 level、iter；根用 $\phi = \alpha r$。

節點	rank	父 rank：前→後	level：前→後	iter：前→後	ϕ：前→後
14	1	2→6	0→1	1→1	2→1
12	2	5→6	1→1	1→1	3→3
0	5	6→6	0→0	1→1	14→14
32（根）	6	6→6	不適用	不適用	18→18

對 14，壓縮前 $A_0(1) = 2 \le 2 < A_1(1) = 3$，故 level=0、iter=1。壓縮後 $A_1(1) = 3 \le 6 < A_2(1) = 7$，且 $A_1^{(2)}(1) = 7 > 6$，故 level=1、iter=1。位能由 $(3-0)1 - 1 = 2$ 降至 $(3-1)1 - 1 = 1$。

對 12，兩次皆有 $5 = A_1(2) \le p < 11 = A_1^{(2)}(2) < A_2(2) = 23$，故 level=1、iter=1。對 0，$A_0(5) = 6 \le 6 < A_0^{(2)}(5) = 7 < A_1(5) = 11$，故 level=0、iter=1、位能 14。路徑總和 $37 \to 36$；其餘點的 rank、父 rank 與根身分不變，故**全森林位能也恰減 1**。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

好節點證明：每一步不等式各用了什麼？

所有量先固定在壓縮前。記 $\rho(v) = \mathrm{rank}(v)$、$p_x = \mathrm{parent}(x)$、$p_y = \mathrm{parent}(y)$，根為 R。令 $r = \rho(x)$、$k = \ell(x) = \ell(y)$、$i = i(x)$。

逐步推導	理由
$\rho(R) \geq \rho(p_y)$	p_y 在通往根的路上，rank 沿非根 parent 邊嚴格增加
$\rho(p_y) \geq A_k(\rho(y))$	$\ell(y) = k$ 的定義
$A_k(\rho(y)) \geq A_k(\rho(p_x))$	y 就是 p_x 或其祖先，故 $\rho(y) \geq \rho(p_x)$；A_k 遞增
$A_k(\rho(p_x)) \geq A_k(A_k^{(i)}(r))$	iter 定義給 $\rho(p_x) \geq A_k^{(i)}(r)$，再用 A_k 遞增
$A_k(A_k^{(i)}(r)) = A_k^{(i+1)}(r)$	函數再合成一次，不是數值相乘

因此新父的 rank 至少為 $A_k^{(i+1)}(r)$。這裡沒有使用「路徑上的 level 遞增」；只使用 rank 的祖先次序與同一個 k 的函數單調性。

跨過下一個門檻，為何一定能支付工作？

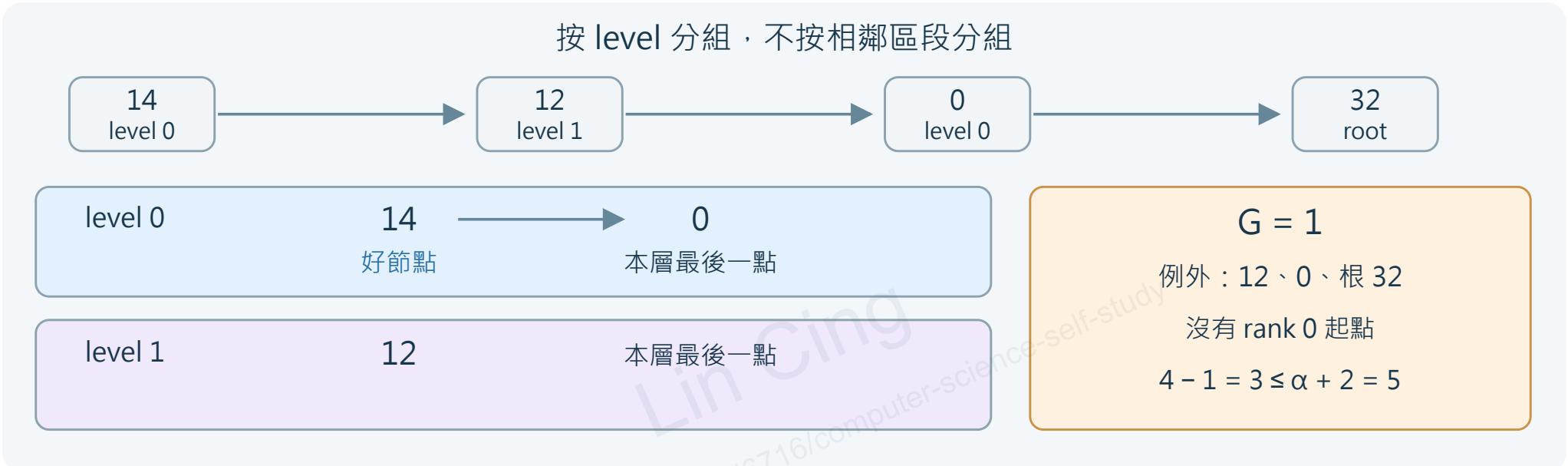
壓縮使 x 的父變成 R，但 r 不變。前頁證明的是新父 rank 跨過**舊 level 的下一次迭代門檻**，不保證新的 iter 必定比舊值大；必須分兩種情況。

壓縮後	位能的差	可以支付的原因
level 仍為 k	$\Delta\phi = -(i' - i) \le -1$	iter 至少變成 $i+1$
level 升為 $k' > k$	$\Delta\phi = -(k'-k)r + i - i'$	$k'-k \ge 1, i \le r, i' \ge 1$，故 $\Delta\phi \le -r + (r-1) = -1$

在 64 點例子取 $x = 14, y = 0, k = 0, i = 1$，不等式數值為 $6 \ge 6 \ge 6 \ge 3 \ge 3$。舊層下一個門檻是 $A_0^{(2)}(1) = 3$；新父 rank=6 更跨過 $A_1(1) = 3$，所以落在第二種情況，位能下降 1。

這筆帳按「被壓縮的好節點 x」計費，不向見證 y 再收一次，也不要求 y 的位能下降。所有非好節點的位能亦不增加，故可以把各個好節點的下降量相加。

每個 level 只留下最後一點



沿原路徑向根看，同一 level 除了最後一點外，每點都有後方的同層見證。因此 14 可計費，12 與 0 分別是 level 1、0 的最後一點；再加根 32，共 3 個例外。若把三個相鄰區段 0／1／0 當成三組，就會漏算好節點 14。

從分組計數回到位能總下降

一般路徑含 s 個節點。rank 大於 0 的非根節點的 level 只能是 $0, \dots, \alpha - 1$，故各層最後一點至多 α 個；另加根與可能的 rank 0 起點，例外至多 $\alpha + 2$。rank 沿路嚴格增加，rank 0 起點最多一個。

$$G \geq g := \max(0, s - (\alpha + 2)), \qquad \Delta\Phi \leq -G \leq -g.$$

G 是實際好節點數，g 只是保守下界。64 點例子 $s = 4, \alpha = 3$，所以 $g = 0$，實際卻有 $G = 1$、$\Delta\Phi = -1$。短路徑的下界可以不緊；不等式並未要求實際下降量必須等於下界。後面會用這個一般下界支付長路徑的成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用固定常數把路徑工作付清

設兩趟查找的實際成本至多 cs，固定 $C > c$ 並用放大位能 $\Psi = C\Phi$。則攤銷成本

$$\hat{c} = c_{\text{actual}} + \Delta\Psi \le cs - Cg.$$

路徑情況	代入 g 後的上界
$s \le \alpha + 2$	$cs \le c(\alpha + 2)$
$s > \alpha + 2$	$(c - C)s + C(\alpha + 2) \le C(\alpha + 2)$

因此每次 FIND 的攤銷成本為 $O(1 + \alpha)$。例如 $s = 7$、$\alpha = 3$、$c = 2$、$C = 3$，下界 $g = 2$，故 $\hat{c} \le 14 - 6 = 8$。C 在整段分析中固定，不能每次臨時換值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LINK 把位能增加限制在新根

MAKE-SET 新增 rank 0 節點，其位能為 0，攤銷成本仍為 $O(1)$。對 LINK，設舊根 u 接到保留根 v：

可能改變的點	位能變化的原因
u	從根變非根；rank 不變，位能不增
v 原有的孩子	父 rank 不降；前述單調性保證位能不增
v	不同 rank 時不變；同 rank 時加一，位能只增 α

其他點的 rank、父 rank 與根身分皆不變，所以位能不變。於是 $\Delta\Phi \le C\alpha$，LINK 的攤銷成本為 $O(1) + C\alpha = O(1+\alpha)$。UNION 先做兩次 FIND，根不同才 LINK；最多拆成三次操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從每次攤銷界回到整段實際成本

令第 j 次實際成本為 c_j，攤銷成本為 $a_j = c_j + \Psi_j - \Psi_{j-1}$。相鄰位能相消後，

$$\sum_{j=1}^{m'} c_j = \sum_{j=1}^{m'} a_j + \Psi_0 - \Psi_{m'} \le \sum_{j=1}^{m'} a_j.$$

不等號用了初始空森林 $\Psi_0 = 0$ 與終態位能非負。拆開 UNION 後 $m' \le 3m$，所以原本 m 次操作的總成本為 $O(m(1+\alpha(n)))$。

這是整段序列的最壞總界。rank 與壓縮並用時，單次 FIND 仍可能走 $\Theta(\log n)$；先建立平衡的 rank 樹、再首次查找深葉即可達到。線性長鏈需要移除 rank 規則，不能當成本定理的單次反例。

位能分析參考：Cormen 等，第三版 §21.4，印刷頁 576–581。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study